

MapReduce 初级编程实践

1. 实验目的

- (1) 通过实验掌握基本的 MapReduce 编程方法;
- (2) 掌握用 MapReduce 解决一些常见的数据处理问题, 包括数据去重、数据排序和数据挖掘等。

2. 实验步骤

(一) 编程实现文件合并和去重操作

对于两个输入文件, 即文件 A 和文件 B, 请编写 MapReduce 程序, 对两个文件进行合并, 并剔除其中重复的内容, 得到一个新的输出文件 C。下面是输入文件和输出文件的一个样例供参考。

输入文件 A 的样例如下:

20170101	x
20170102	y
20170103	x
20170104	y
20170105	z
20170106	x

输入文件 B 的样例如下:

20170101	y
20170102	y
20170103	x
20170104	z
20170105	y

根据输入文件 A 和 B 合并得到的输出文件 C 的样例如下:

20170101	x
20170101	y
20170102	y
20170103	x
20170104	y
20170104	z
20170105	y
20170105	z

（二）编写程序实现对输入文件的排序

现在有多个输入文件，每个文件中的每行内容均为一个整数。要求读取所有文件中的整数，进行升序排序后，输出到一个新的文件中，输出的数据格式为每行两个整数，第一个数字为第二个整数的排序位次，第二个整数为原待排列的整数。下面是输入文件和输出文件的一个样例供参考。

输入文件 1 的样例如下：

```
33
37
12
40
```

输入文件 2 的样例如下：

```
4
16
39
5
```

输入文件 3 的样例如下：

```
1
45
25
```

根据输入文件 1、2 和 3 得到的输出文件如下：

```
1 1
2 4
3 5
4 12
5 16
6 25
7 33
8 37
9 39
10 40
11 45
```

（三）对给定的表格进行信息挖掘

下面给出一个 **child-parent** 的表格，要求挖掘其中的父子辈关系，给出祖孙辈关系的表格。

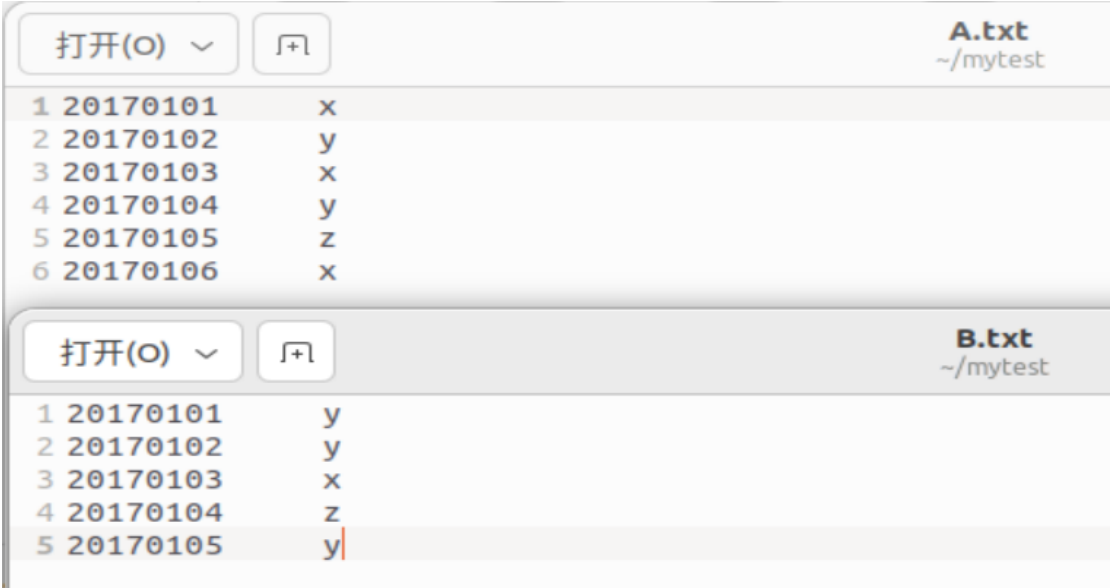
输入文件内容如下：

child	parent
Steven	Lucy
Steven	Jack
Jone	Lucy
Jone	Jack
Lucy	Mary
Lucy	Frank
Jack	Alice
Jack	Jesse
David	Alice
David	Jesse
Philip	David
Philip	Alma
Mark	David
Mark	Alma

输出文件内容如下：

grandchild	grandparent
Steven	Alice
Steven	Jesse
Jone	Alice
Jone	Jesse
Steven	Mary
Steven	Frank
Jone	Mary
Jone	Frank
Philip	Alice
Philip	Jesse
Mark	Alice
Mark	Jesse

4.实验报告

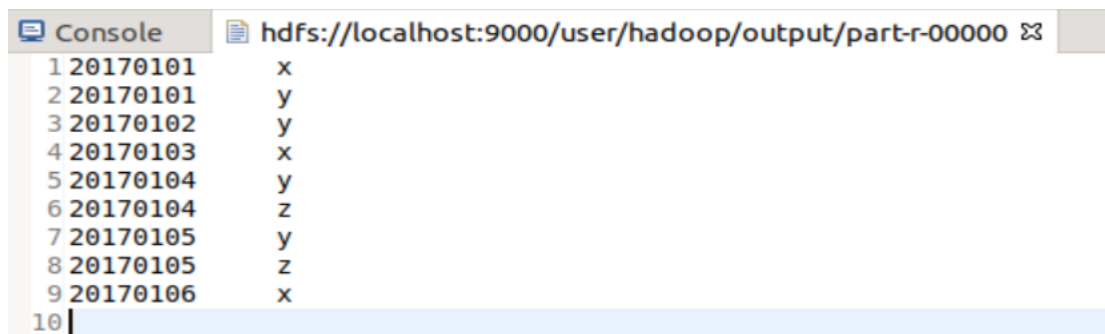
实验报告			
姓名	李聪	日期	2023/11/28
实验环境	操作系统: Ubuntu 22.04 Hadoop 版本: 2.7.1 Java IDE: Eclipse		
实验内容与完成情况:			
(一) 编程实现文件合并和去重操作			
1.1 删除 input 文件夹下的所有文件			
\$ <code>hadoop fs -rm -r /user/hadoop/input/*</code>			
1.2 删除 output 文件夹			
\$ <code>hadoop fs -rm -r output</code>			
1.3 在本地创建 A.txt 和 B.txt 两个文件			
\$ <code>vim A.txt</code>			
\$ <code>vim B.txt</code>			
			
1.4 将 A.txt 和 B.txt 两个文件上传至 hadoop			
\$ <code>hadoop fs -copyFromLocal ~/mytest/A.txt /user/hadoop/input</code>			
\$ <code>hadoop fs -copyFromLocal ~/mytest/B.txt /user/hadoop/input</code>			
1.5 编写程序			

```

public class MapReduce1 {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        String[] otherArgs = new String[] {"input", "output"};
        if(otherArgs.length != 2) {
            System.err.println("Usage: MapReduce1 <in> [<in>...] <out>");
            System.exit(2);
        }
        Job job = Job.getInstance(conf, "MapReduce1");
        job.setJarByClass(MapReduce1.class);
        job.setMapperClass(MapReduce1.Map.class);
        job.setReducerClass(MapReduce1.Reduce.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);
        FileInputFormat.addInputPath(job, new Path(otherArgs[0]));
        FileOutputFormat.setOutputPath(job, new Path(otherArgs[1]));
        System.exit(job.waitForCompletion(true)?0:1);
    }
    public static class Map extends Mapper<Object, Text, Text, Text>{
        private static Text text = new Text();
        public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {
            text = value;
            context.write(text, new Text(""));
        }
    }
    public static class Reduce extends Reducer<Text, Text, Text, Text>{
        public void reduce(Text key, Iterable<Text> values, Context context)
            throws IOException, InterruptedException {
            context.write(key, new Text(""));
        }
    }
}

```

1.6 运行结果



```

Console hdfs://localhost:9000/user/hadoop/output/part-r-00000
1 20170101 x
2 20170101 y
3 20170102 y
4 20170103 x
5 20170104 y
6 20170104 z
7 20170105 y
8 20170105 z
9 20170106 x
10

```

(二) 编写程序实现对输入文件的排序

2.1 删除 input 文件夹下的所有文件

```
$ hadoop fs -rm -r /user/hadoop/input/*
```

2.2 删除 output 文件夹

```
$ hadoop fs -rm -r output
```

2.3 在本地创建 1.txt、2.txt 和 3.txt 三个文件

```
$ vim 1.txt
```

```
$ vim 2.txt
```

```
$ vim 3.txt
```



2.4 将 1.txt、2.txt 和 3.txt 三个文件上传至 hadoop

```
$ hadoop fs -copyFromLocal ~/mytest/1.txt /user/hadoop/input
```

```
$ hadoop fs -copyFromLocal ~/mytest/2.txt /user/hadoop/input
```

```
$ hadoop fs -copyFromLocal ~/mytest/2.txt /user/hadoop/input
```

2.5 编写程序

```
public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    String[] otherArgs = new String[] {"input", "output"};
    if(otherArgs.length != 2) {
        System.err.println("Usage: MapReduce1 <in> [<in>...] <out>");
        System.exit(2);
    }
    Job job = Job.getInstance(conf, "MapReduce2");
    job.setJarByClass(MapReduce2.class);
    job.setMapperClass(MapReduce2.Map.class);
    job.setReducerClass(MapReduce2.Reduce.class);
    job.setPartitionerClass(MapReduce2.Partition.class);
    job.setOutputKeyClass(IntWritable.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(otherArgs[0]));
    FileOutputFormat.setOutputPath(job, new Path(otherArgs[1]));
    System.exit(job.waitForCompletion(true)?0:1);
}
```

```

public static class Map extends Mapper<Object, Text, IntWritable, IntWritable>{
    private static IntWritable data = new IntWritable();
    public void map(Object key, Text value, Context context) throws IOException, InterruptedException{
        String text = value.toString();
        data.set(Integer.parseInt(text));
        context.write(data, new IntWritable(1));
    }
}
public static class Partition extends Partitioner<IntWritable, IntWritable>
{
    public int getPartition(IntWritable key, IntWritable value, int n) {
        int MAX = 65223;
        int bound = MAX / n + 1;
        int keyNum = key.get();
        for(int i = 0; i < n; ++ i) {
            if(keyNum < bound * (i + 1) && keyNum >= bound * i)
            {
                return i;
            }
        }
        return -1;
    }
}
public static class Reduce extends Reducer<IntWritable, IntWritable, IntWritable, IntWritable>{
    private static IntWritable n = new IntWritable(1);
    public void reduce(IntWritable key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException{
        for (IntWritable val : values) {
            context.write(n, key);
            n = new IntWritable(n.get() + 1);
        }
    }
}
}

```

2.6 运行结果

Console hdfs://localhost:9000/user/hadoop/output/part-r-00000

```

1 1 1
2 2 4
3 3 5
4 4 12
5 5 16
6 6 25
7 7 33
8 8 37
9 9 39
10 10 40
11 11 45
12 |

```

(三) 对给定的表格进行信息挖掘

3.1 删除 input 文件夹下的所有文件

```
$ hadoop fs -rm -r /user/hadoop/input/*
```

3.2 删除 output 文件夹

```
$ hadoop fs -rm -r output
```

3.3 在本地创建 child-parent.txt 文件

```
$ vim child-parent.txt
```

打开(O) ▾

+

*child-parent.txt
~/mytest

```
1 child parent
2 Steven Lucy
3 Steven Jack
4 Jone Lucy
5 Jone Jack
6 Lucy Mary
7 Lucy Frank
8 Jack Alice
9 Jack Jesse
10 David Alice
11 David Jesse
12 Philip David
13 Philip Alma
14 Mark David
15 Mark Alma
```

3.4 将 child-parent.txt 文件上传至 hadoop

```
$ hadoop fs -copyFromLocal ~/mytest/child-parent.txt /user/hadoop/input
```

3.5 编写程序

```
public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    String[] otherArgs = new String[] {"input", "output"};
    if(otherArgs.length != 2) {
        System.err.println("Usage: MapReduce1 <in> [<in>...] <out>");
        System.exit(2);
    }
    Job job = Job.getInstance(conf, "MapReduce3");
    job.setJarByClass(MapReduce3.class);
    job.setMapperClass(MapReduce3.Map.class);
    job.setReducerClass(MapReduce3.Reduce.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);
    FileInputFormat.addInputPath(job, new Path(otherArgs[0]));
    FileOutputFormat.setOutputPath(job, new Path(otherArgs[1]));
    System.exit(job.waitForCompletion(true)?0:1);
}

public static class Map extends Mapper<Object, Text, Text, Text>{
    private static IntWritable data = new IntWritable();
    public void map(Object key, Text value, Context context) throws IOException, InterruptedException{
        String childName = new String();
        String parentName = new String();
        String relationType = new String();
        String line = value.toString();

        int i = 0;
        while(line.charAt(i) != ' '){
            i++;
        }

        String[] values = {line.substring(0, i), line.substring(i + 1)};
        if(values[0].compareTo("child") != 0) {
            childName = values[0];
            parentName = values[1];
            relationType = "1";
            context.write(new Text(values[1]), new Text(relationType + "+" + childName + "+" + parentName));
            relationType = "2";
            context.write(new Text(values[0]), new Text(relationType + "+" + childName + "+" + parentName));
        }
    }
}
```



```

public static int flag = 0;
public static class Reduce extends Reducer<Text, Text, Text, Text>{
    public void reduce(Text key, Iterable<Text> values, Context context)
        throws IOException, InterruptedException {
        if(flag == 0) {
            context.write(new Text("grandchild"), new Text("grandparent"));
            flag ++;
        }
        int grandChildNum = 0;
        String grandChild[] = new String[10];
        int grandParentNum = 0;
        String grandParent[] = new String[10];
        Iterator ite = values.iterator();
        while(ite.hasNext()) {
            String record = ite.next().toString();
            int len = record.length();
            int i = 2;
            if(len == 0) continue;

            char relationType = record.charAt(0);
            String childName = new String();
            String parentName = new String();
            while(record.charAt(i) != '+') {
                childName = childName + record.charAt(i);
                i ++;
            }
            i = i + 1;
            while(i < len) {
                parentName = parentName + record.charAt(i);
                i ++;
            }
            if(relationType == '1') {
                grandChild[grandChildNum] = childName;
                grandChildNum ++;
            }
            else {
                grandParent[grandParentNum] = parentName;
                grandParentNum ++;
            }
        }
        if(grandParentNum != 0 && grandChildNum != 0) {
            for(int m = 0; m < grandChildNum; m++) {
                for(int n = 0; n < grandParentNum; n++) {
                    context.write(new Text(grandChild[m]), new Text(grandParent[n]));
                }
            }
        }
    }
}

```

3.6 运行结果

MapReduce3.java hdfs://localhost:9000/user/hadoop/output/part-r-000000

```

1 grandchild grandparent
2 Mark      Jesse
3 Mark      Alice
4 Philip    Jesse
5 Philip    Alice
6 Jone      Jesse
7 Jone      Alice
8 Steven    Jesse
9 Steven    Alice
10 Steven   Frank
11 Steven   Mary
12 Jone     Frank
13 Jone     Mary
14

```

出现的问题:

1. 找不到 Eclipse 安装的位置

解决方案（列出遇到的问题和解决办法，列出没有解决的问题）:

1. 卸载重装